

Evaluación Práctica: Sistema de Gestión de Personal con Django

1. Situación de evaluación

Los estudiantes, de manera **individual**, deberán ampliar la aplicación web desarrollada previamente en Django, la cual actualmente cuenta con:

- Registro de usuarios.
- Inicio de sesión.
- Cierre de sesión.
- Página principal o Home.
- Manejo de sesiones mediante el sistema de autenticación de Django.

A partir de esta aplicación, deberán implementar un nuevo módulo orientado a la **administración de datos del personal de una empresa**, utilizando Django Admin, Django ORM y operaciones CRUD.

Durante el desarrollo podrán utilizar herramientas de **Inteligencia Artificial como apoyo para la estructuración del código, generación de interfaces, resolución de errores y mejora de la solución**, siendo responsabilidad del estudiante comprender y adaptar correctamente el código incorporado.

Evidencia requerida

Aplicación web desarrollada con Django que implemente Django Admin y operaciones CRUD para la gestión de empleados, cargos y departamentos.

2. Contexto

Una empresa requiere ampliar su sistema interno para incorporar un módulo que permita administrar la información básica de sus trabajadores.

Actualmente, el sistema ya permite que los usuarios se registren e inicien sesión.

La empresa solicita incorporar al Home una nueva opción denominada:

Administración de Personal

Desde esta opción, los usuarios autenticados deberán poder acceder a un módulo que permita administrar:

- Empleados.
- Cargos.
- Departamentos.

La información deberá almacenarse en la base de datos utilizando los modelos y herramientas proporcionadas por Django.

3. Objetivo de la evaluación

Desarrollar un módulo web que permita gestionar información del personal de una empresa, aplicando:

- Modelos de Django.
- Relaciones entre modelos.
- Django ORM.
- Django Admin.
- Operaciones CRUD.
- Formularios.
- Templates.
- Manejo de sesiones y autenticación.
- Herramientas de Inteligencia Artificial como apoyo al desarrollo.

4. Modelos requeridos

El sistema deberá incorporar como mínimo los siguientes modelos:

Departamento

Representa las diferentes áreas existentes dentro de la empresa.

Deberá contener como mínimo:

- Nombre.
- Descripción.

Ejemplos:

- Informática.
- Recursos Humanos.
- Finanzas.
- Operaciones.
- Administración.

Cargo

Representa los distintos cargos que pueden desempeñar los trabajadores.

Deberá contener como mínimo:

- Nombre del cargo.
- Descripción.

Ejemplos:

- Analista.
- Supervisor.
- Ingeniero.
- Técnico.
- Administrativo.
- Jefe de Departamento.

Empleado

Representa a cada trabajador perteneciente a la empresa.

Deberá contener como mínimo:

- RUT o identificador.
- Nombre.
- Apellido.
- Correo electrónico.
- Teléfono.
- Fecha de ingreso.
- Cargo.
- Departamento.
- Estado.

El campo Cargo deberá establecer una relación con el modelo Cargo.

El campo Departamento deberá establecer una relación con el modelo Departamento.

El estado podrá considerar, por ejemplo:

- Activo.
- Inactivo.

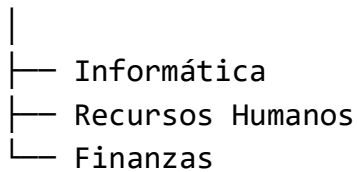
El estudiante podrá agregar otros atributos si lo considera necesario.

5. Relaciones entre los modelos

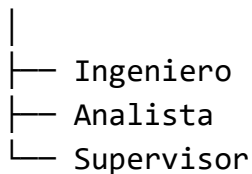
La aplicación deberá implementar correctamente las relaciones mediante Django ORM.

La estructura general deberá permitir representar situaciones como:

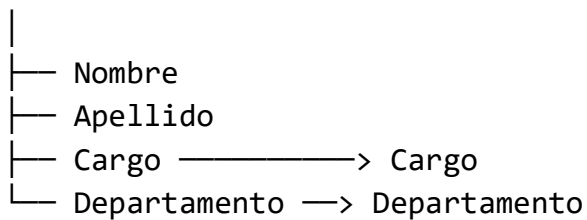
Departamento



Cargo



Empleado



Por ejemplo:

Empleado: Juan Pérez

Cargo: Ingeniero

Departamento: Informática

Las relaciones deberán implementarse utilizando las funcionalidades proporcionadas por Django.

6. Integración con el Home

La aplicación existente deberá ser modificada para incorporar desde el Home un acceso al nuevo módulo.

Por ejemplo:

Administración de Personal

Al ingresar, el usuario deberá visualizar una pantalla principal del módulo.

Esta podrá incluir accesos como:

- Ver empleados.
- Registrar empleado.
- Administrar cargos.
- Administrar departamentos.

La navegación deberá permitir regresar al Home principal.

7. Gestión mediante Django Admin

Los modelos:

- Empleado.
- Cargo.
- Departamento.

deberán encontrarse registrados correctamente dentro de **Django Admin**.

Desde el administrador deberá ser posible realizar operaciones sobre los tres modelos.

Como mínimo:

- Crear registros.
- Visualizar registros.
- Modificar registros.
- Eliminar registros.

Además, el estudiante deberá personalizar al menos uno de los modelos administrados utilizando opciones como:

- `list_display`
- `search_fields`
- `list_filter`
- `ordering`

Por ejemplo, para el modelo Empleado se podría visualizar:

RUT | Nombre | Apellido | Cargo | Departamento | Estado

También podría incorporarse búsqueda por:

- Nombre.
- Apellido.
- RUT.

Y filtros por:

- Departamento.
- Cargo.
- Estado.

8. CRUD de empleados

Además de Django Admin, la aplicación deberá implementar una **interfaz propia** para administrar empleados.

El módulo deberá permitir realizar las cuatro operaciones fundamentales:

Crear

Registrar un nuevo empleado.

El formulario deberá permitir seleccionar:

- Cargo.
- Departamento.



- Estado.

Los cargos y departamentos deberán obtenerse desde la base de datos.

Leer

Visualizar los empleados registrados.

Como mínimo deberán mostrarse:

- RUT.
- Nombre completo.
- Cargo.
- Departamento.
- Estado.

Actualizar

Permitir modificar la información de un empleado existente.

Al ingresar a la opción de edición, el formulario deberá cargar previamente la información registrada.

Eliminar

Permitir eliminar un empleado.

Antes de realizar la eliminación se deberá solicitar confirmación al usuario.

Por ejemplo:

¿Está seguro de que desea eliminar al empleado Juan Pérez?

9. Vista de detalle

Cada empleado deberá disponer de una vista que permita consultar su información completa.

Por ejemplo:

INFORMACIÓN DEL EMPLEADO

RUT: 12.345.678-9

Nombre: Juan Pérez

Correo: jperez@empresa.cl

Teléfono: +56 9 1234 5678

Cargo: Ingeniero de Sistemas

Departamento: Informática

Fecha ingreso: 10/03/2024

Estado: Activo

10. Gestión de cargos y departamentos

Los cargos y departamentos deberán poder administrarse mediante **Django Admin**.

Como mínimo el estudiante deberá ser capaz de:

Departamento

- Crear departamentos.
- Modificar departamentos.
- Eliminar departamentos.
- Visualizar departamentos registrados.

Cargo

- Crear cargos.
- Modificar cargos.
- Eliminar cargos.
- Visualizar cargos registrados.

Estos registros deberán utilizarse posteriormente en los formularios de empleados.

11. Uso de Django ORM

Todas las operaciones de acceso a datos deberán realizarse utilizando **Django ORM**.

Por ejemplo:

```
Empleado.objects.all()
Empleado.objects.get()
Empleado.objects.filter()
```

También podrán utilizarse las funcionalidades proporcionadas por:

- `ModelForm`.
- `ListView`.
- `CreateView`.

- UpdateView.
- DeleteView.
- DetailView.

El estudiante podrá utilizar vistas basadas en funciones o vistas basadas en clases.

12. Manejo de sesiones

El módulo de administración de personal deberá integrarse con el sistema de autenticación desarrollado previamente.

Solo los usuarios autenticados podrán ingresar al módulo.

Un usuario que no haya iniciado sesión no deberá poder acceder directamente a las páginas de administración de empleados.

Por ejemplo:

`/login/`

Luego de autenticarse, podrá acceder a:

`/home/
personal/
personal/empleados/`

Las acciones de:

- Crear.
- Modificar.
- Eliminar.

deberán encontrarse protegidas para usuarios autenticados.

13. Interfaces requeridas

La aplicación deberá contar como mínimo con las siguientes interfaces:

1. Home existente.
2. Página principal del módulo de personal.
3. Listado de empleados.
4. Registro de empleado.
5. Detalle de empleado.
6. Modificación de empleado.
7. Confirmación de eliminación.

Las interfaces deberán desarrollarse utilizando:

- Django Templates.
- HTML.
- CSS.

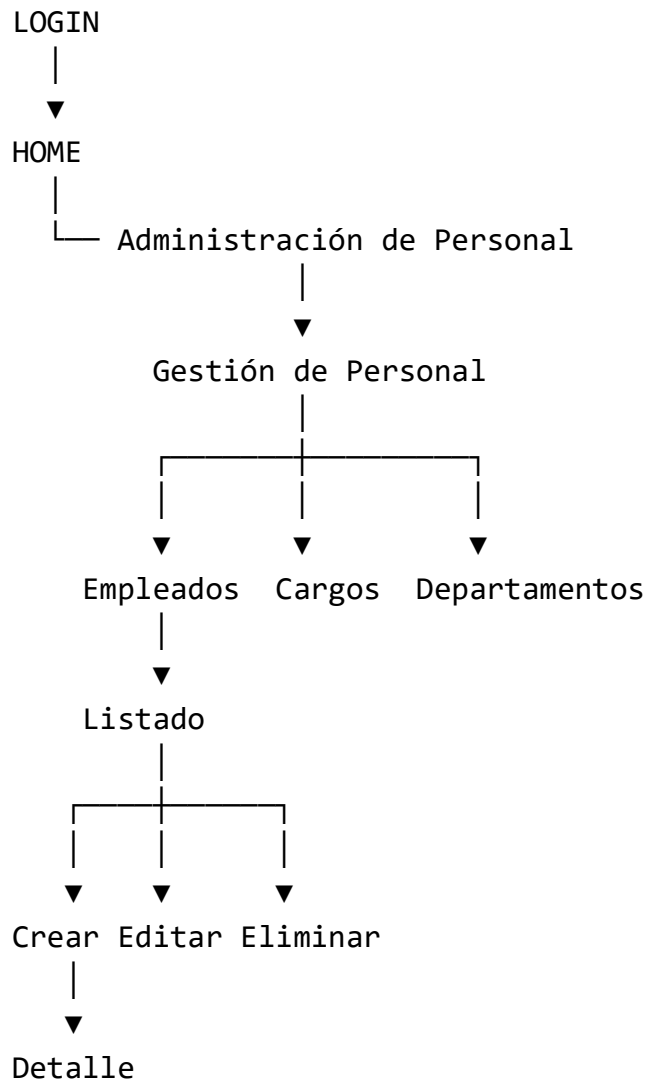
Opcionalmente podrán utilizar:

- Bootstrap.
- Tailwind.
- Otra biblioteca de estilos.

El diseño deberá mantener coherencia con la aplicación desarrollada previamente.

14. Ejemplo de navegación

Una posible estructura de navegación podría ser:



15. Uso de Inteligencia Artificial

Durante la evaluación se permitirá utilizar Inteligencia Artificial como herramienta de apoyo.

Podrá utilizarse para:

- Diseñar modelos.
- Analizar relaciones entre modelos.
- Crear formularios.
- Generar templates HTML.
- Mejorar el diseño visual.
- Detectar errores.
- Proponer soluciones.
- Explicar código.
- Refactorizar código.

El uso de IA deberá estar orientado a **apoyar el aprendizaje y desarrollo**, no simplemente a copiar una solución completa.

El estudiante deberá ser capaz de explicar el código desarrollado.

Durante la revisión el docente podrá seleccionar cualquier parte de la aplicación y solicitar una explicación de su funcionamiento.

16. Evidencias mínimas de funcionamiento

Para considerar la evaluación completa, deberá ser posible demostrar el siguiente flujo:

1. Iniciar sesión.
2. Ingresar al Home.
3. Acceder a Administración de Personal.
4. Visualizar empleados existentes.
5. Registrar un nuevo empleado.
6. Seleccionar un cargo existente.
7. Seleccionar un departamento existente.
8. Guardar el empleado.
9. Visualizar el nuevo empleado en el listado.
10. Consultar el detalle del empleado.
11. Modificar algún dato.
12. Guardar la modificación.
13. Eliminar un empleado.
14. Confirmar la eliminación.
15. Administrar cargos y departamentos desde Django Admin.

17. Requerimientos mínimos

Para aprobar correctamente la actividad deberá evidenciarse:

- Conexión funcional con la base de datos mediante Django.
- Modelo Empleado.
- Modelo Cargo.
- Modelo Departamento.
- Relaciones entre los modelos.
- Migraciones correctamente realizadas.
- Uso de Django ORM.
- Configuración de Django Admin.
- CRUD completo de empleados.
- Formularios funcionales.
- Templates funcionales.
- Integración con el sistema de autenticación.
- Restricción de acceso mediante sesión.
- Navegación desde el Home.
- Uso pertinente de Inteligencia Artificial como apoyo.

18. Entrega

El estudiante deberá entregar el proyecto Django completo.

La aplicación deberá ejecutar correctamente mediante:

```
python manage.py runserver
```

Las migraciones deberán encontrarse incluidas dentro del proyecto.

Se deberá entregar todo el código necesario para poder ejecutar y revisar la aplicación.

19. Criterios de evaluación

Criterio	Puntaje
Creación correcta de los modelos Empleado, Cargo y Departamento (o similares)	10 pts
Relaciones correctamente implementadas entre los modelos	10 pts
Uso correcto de Django ORM	10 pts
Configuración y registro de modelos en Django Admin	10 pts
Personalización de Django Admin	5 pts
Integración del módulo con el Home existente	5 pts
Listado de empleados	5 pts
Registro de empleados	10 pts
Modificación de empleados	10 pts
Eliminación de empleados con confirmación	10 pts
Vista de detalle del empleado	5 pts
Integración con autenticación y manejo de sesiones	5 pts
Uso adecuado de templates e interfaz	3 pts
Uso pertinente de Inteligencia Artificial	2 pts
TOTAL	100 pts

20. Validación individual

Durante la revisión, el docente podrá solicitar al estudiante realizar una modificación sencilla en su proyecto.

Por ejemplo:

- Agregar un nuevo campo al modelo.
- Incorporar un filtro.
- Modificar una consulta ORM.
- Agregar una columna al Django Admin.
- Modificar un formulario.
- Crear un nuevo departamento.
- Cambiar un empleado de cargo.
- Cambiar un empleado de departamento.
- Explicar una relación entre modelos.



El estudiante deberá ser capaz de identificar dónde realizar el cambio y explicar el funcionamiento general de la solución desarrollada.